

PHASE 1

CREATING A CUSTOM APPLICATION

Establishing the isolated scoped application that will contain every table, script, catalog item, portal page, and widget built in this project.

Employee Raise Issue — ServiceNow Implementation Guide

Standalone Phase Document 1 of 7

Document Version 1.0

About This Document

This document is a standalone, self-contained implementation guide for Phase 1 — Creating a Custom Application — of the Employee Raise Issue project. It can be read and actioned on its own, without needing the full multi-phase master documentation, though it references the wider project for context where useful.

Field	Detail
Application	Employee Raise Issue (built within the Employee Center scope)
Phase	Phase 1 of 7
Primary Owner	Platform / ServiceNow Administrator
Related Document	Employee Raise Issue Documentation (full 7-phase project reference)
Instance Referenced	dev218486.service-now.com
Document Version	1.0

Contents

1. Purpose and Scope
2. Where This Phase Fits in the Project
3. Objective
4. Why a Scoped Application Matters
5. Prerequisites
 - 5.1 Planning Worksheet
6. Step-by-Step Configuration
7. Screenshot Reference
8. Application-Level Settings
9. Naming Conventions Reference
10. Security and Access Considerations
11. Common Issues and Troubleshooting
12. Validating Phase 1 Completion
13. Best Practices
 - 13.1 Frequently Asked Questions
14. Summary and Handoff to Phase 2
 - 14.1 Phase 1 Deliverables Checklist

1. Purpose and Scope

This document is the complete, standalone implementation guide for Phase 1 of the Employee Raise Issue project: creating the custom application that will act as the container for every other component built in this project. It is written for an administrator who needs to either build this phase from scratch on a new instance, or understand and verify how it was built on the working development instance (dev218486).

By the end of this phase, a working, correctly scoped application exists on the instance, ready to receive the custom table, business logic, Record Producer, Service Portal pages, and widgets documented in Phases 2 through 7.

2. Where This Phase Fits in the Project

The Employee Raise Issue project is delivered across seven implementation phases. Phase 1 is the foundation on which every later phase depends:

1. Phase 1 — Creating a Custom Application (this document)
2. Phase 2 — Creating a Custom Table and Fields
3. Phase 3 — Creating UI Policies and Dependency
4. Phase 4 — Creating a Record Producer
5. Phase 5 — Creating a New Service Portal
6. Phase 6 — Creating Widgets and Adding to the Page
7. Phase 7 — Testing and Validation

Every artifact created in Phases 2–7 — the table, choice lists, UI Policies, Record Producer, portal pages, and widgets — lives inside the application scope established here. Getting this phase right matters disproportionately: a poorly chosen or inconsistent scope makes every later phase harder to maintain, and moving components between scopes later is a data migration exercise, not a simple setting change.

Phase	Focus	Depends On This Phase Because...
1	Creating a Custom Application (this document)	N/A — this is the foundation
2	Custom Table and Fields	The table must be created inside the correct application scope
3	UI Policies and Dependency	Policies are defined against the scoped table from Phase 2
4	Record Producer	The producer is filed under, and points to a table within, this scope
5	New Service Portal	Portal pages and menus are created inside the same scope
6	Widgets	Widgets are packaged and versioned as part of this application
7	Testing and Validation	Test scope, roles, and ACLs are all defined relative to this application

3. Objective

Establish a dedicated, isolated application container — Employee Raise Issue — so that every table, script, Record Producer, portal page, and widget built during this project has a clean, well-understood home. Using a scoped application (rather than the Global scope) is the ServiceNow-recommended approach for any custom solution, because it isolates customizations, simplifies future upgrades, and provides a clean boundary for access control.

Concretely, this phase should leave you with an application record that is active, correctly named, discoverable by other administrators, and ready to accept new tables and configuration.

4. Why a Scoped Application Matters

It is possible to build tables, business rules, and UI Policies directly in the Global scope. ServiceNow will not stop you. However, doing so on anything beyond a trivial, single-table proof of concept creates real, compounding problems as the solution grows. The table below summarizes why this project uses a dedicated scope.

Consideration	Global Scope	Dedicated Scoped Application
Naming collisions	Table and field names share one flat namespace with every other customization on the instance.	Table and field names are automatically prefixed and isolated from unrelated customizations.
Upgrade safety	Global customizations are more exposed to being affected by platform upgrades and skip-level patches.	Scoped applications are explicitly designed to survive upgrades cleanly.
Packaging and portability	Difficult to cleanly export just "this project" for migration to another instance.	The entire application — tables, scripts, portal pages, widgets — can be exported and installed as one unit.
Access control boundary	Any script with sufficient role can touch Global-scope tables and logic.	Cross-scope access is governed explicitly, giving a much clearer security boundary.
Ease of removal	Manually identifying every artifact to uninstall is error-prone.	Deactivating or uninstalling the application removes (or clearly flags) everything it owns.

NOTE: *If your organization already has a suitable employee-facing application (as was the case on the working instance, where Employee Center already existed), it is often reasonable to build inside that existing scope rather than creating a brand-new one — see section 8 and the note under section 6 for how that decision was made on this project.*

5. Prerequisites

Before starting this phase, confirm the following are in place:

- The admin role, or the ability to request scoped application creation from an instance administrator.
- A clear, unique application name and scope prefix agreed upon in advance (for example `x_snc_emp_issue`), unless building inside an existing, already-approved application scope.
- Access to Studio, or to System Applications > Studio from the application navigator.
- Agreement from stakeholders (HR, IT Support) on the application's name and purpose, since this name will appear in the scope picker for every administrator on the instance going forward.

5.1 Planning Worksheet

It is worth completing the following worksheet before opening Studio, so the application is created correctly the first time:

Item	Value to Decide
Application Name	Employee Raise Issue (or the name of the existing application you will build inside)
Scope Prefix	e.g. x_snc_emp_issue — short, unique, unlikely to collide with other vendor scopes
Description	One or two sentences explaining the application's purpose, for future administrators
Owner	The individual or team accountable for the application record
Planned Tables	Employee Issue (built in Phase 2) — list any additional tables anticipated later
Update Set / Source Control	Confirm whether this instance uses Update Sets, Git-backed source control, or both

6. Step-by-Step Configuration

1. Navigate to All > System Applications > Studio, or open Studio directly from the Now Platform application navigator.
2. Select Create Application, then choose Create Application from the Studio launch options.
3. If you are instead building inside an existing application (as on the working instance), select Edit Application and choose the existing application from the list rather than creating a new one, then skip to step 8.
4. Provide the application name: Employee Raise Issue.
5. Provide a short, meaningful scope identifier, for example: x_snc_emp_issue. ServiceNow automatically prefixes this with the vendor code configured on the instance.
6. Add a description that explains the purpose of the application, for example: "Captures and manages employee-raised workplace issues submitted through the Service Portal."
7. Leave the version at the default (1.0.0) for the initial build.
8. Select Create to provision the scoped application record and open the Studio workspace for it.
9. Once Studio opens, confirm the application scope indicator in the top-right corner of the platform (the globe icon) shows the correct application as active before creating any further artifact.
10. Verify the application record was created by navigating to System Applications > Applications > My Company Applications and confirming the application appears in the list with a status of In Development.
11. Bookmark or note the application's sys_id, since you will reference it when configuring Update Sets or cross-application relationships later.

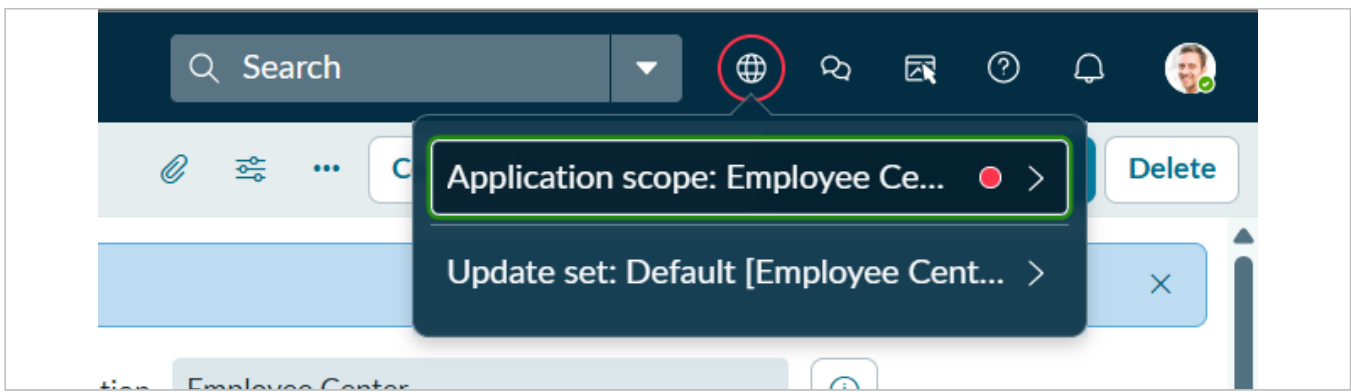


Figure 1 — Application scope selector confirming the active scoped application (Employee Center) on the working instance.

7. Screenshot Reference

The figure above shows the application scope indicator as it appears in the top navigation bar. This is the single most important indicator to check before creating any table, script, or portal artifact — every new component you create is filed under whatever scope is shown here at the time of creation.

NOTE: In this build, the components were developed inside the existing Employee Center application scope rather than a brand-new dedicated scope, since Employee Center already provided the Service Portal, theme, and menu structure needed. The design principles in this phase still apply in full: confirm the correct scope is selected before creating any table, script, or portal artifact, whether that scope is newly created or already exists.

8. Application-Level Settings

Once the application shell exists (or once you have confirmed you are working inside the correct existing application), a small number of application-level settings should be reviewed before moving on to table design in Phase 2:

- Application Menu / Module — decide whether the application should expose a navigator menu for administrators to manage raw records, separate from the employee-facing portal experience.
- Version Control — confirm whether the instance uses Source Control (e.g. Git) for this application, and if so, link the application record to the relevant repository.
- Active flag — leave the application record active so it is available for configuration.
- Application Access — review whether other applications need to read from tables in this scope (Application Access settings), and default to the most restrictive setting that still supports the design.

WATCH OUT: Application scope should not be changed once tables and data begin to accumulate in it. Any restructuring at that point requires a data migration rather than a simple rename — get the scope decision right in this phase, not later.

9. Naming Conventions Reference

Consistent naming makes every later phase easier to build, review, and hand over. The following conventions were used (or should be used, if replicating this build) throughout the Employee Raise Issue application:

Artifact Type	Convention	Example
Table	Descriptive label, lower-case underscored technical name	Employee raises issue / sn_ex_sp_employee_issue

Artifact Type	Convention	Example
Record Producer	Verb + object, Title Case	Raise Employee Issue
Service Portal	Descriptive Title Case name, short URL suffix	Requesting Portal / employee_request
Widget	Descriptive Title Case name matching its on-screen purpose	Commercial Widget, Raise an Issue widget
Business Rule / Script Include	PascalCase for scripts, sentence case for business rule names	EmployeeIssueRouter
Choice values	Human-readable label; lower-case technical value	Forgot Password / forgot_password

10. Security and Access Considerations

Although Phase 1 does not itself create any table or ACL, the scope decision made here directly shapes the security model used in every later phase:

- Roles created for this solution (for example an employee self-service role or an agent role) will be scoped to this application, keeping them clearly identifiable in the role list.
- Access Control Rules (ACLs) created on tables in Phase 2 automatically apply within this scope; cross-scope access from other applications must be explicitly granted.
- If building inside an existing application (as in this project), review that application's current roles and ACLs first, so new rules are additive and consistent rather than conflicting with existing access patterns.

11. Common Issues and Troubleshooting

Issue	Likely Cause	Resolution
"Create Application" option is missing or greyed out	The logged-in user lacks the admin role, or application creation is restricted on this instance.	Confirm the admin role is assigned, or request an administrator create the application shell.
Scope name already exists / is rejected	The chosen scope prefix collides with an existing application or a reserved vendor prefix.	Choose a more specific prefix (add a project or team abbreviation) and retry.
New tables or scripts unexpectedly appear in the wrong application	The application scope indicator was not switched before creating the artifact.	Check the scope indicator before every new artifact; move misplaced artifacts using Studio's "Move to application" option where available, or recreate them in the correct scope.
Studio does not load or times out	Browser cache/session issue, or a temporary platform performance issue.	Clear the browser cache, retry in an incognito window, or check instance health/status.
Application does not appear in My Company Applications	The application was created but not saved correctly, or the list view is filtered.	Clear any active filters on the list, and confirm the create action completed without an error message.

12. Validating Phase 1 Completion

Before moving on to Phase 2, confirm each of the following:

- The application (or the existing application you are building inside) appears in System Applications > Applications > My Company Applications.
- The application status shows In Development (or Active, if already released).
- The application is selectable from the scope picker in the top-right corner of the platform.
- The application description clearly explains its purpose to another administrator opening it for the first time.
- Naming conventions from section 9 are documented and agreed for use in Phases 2–7.

13. Best Practices

BEST PRACTICE: *Keep the application scope short and unambiguous. A confusing or overly generic scope name (e.g. x_snc_app1) makes every later table, script include, and business rule harder to identify during maintenance.*

BEST PRACTICE: *Decide on naming conventions (section 9) before creating the first table, not after — retrofitting naming consistency across dozens of components is far more expensive than agreeing it up front.*

BEST PRACTICE: *If a suitable application scope already exists (as with Employee Center on this project), prefer extending it over creating a parallel scope, provided its existing roles, theme, and menu structure genuinely fit the new solution.*

BEST PRACTICE: *Document the application's owner and purpose in its description field — this is often the only context a future administrator has when deciding whether it is safe to modify or retire.*

13.1 Frequently Asked Questions

Can this application be renamed later?

The display name (label) can be changed at any time without impact. The scope prefix (the technical identifier, e.g. x_snc_emp_issue) should be treated as permanent once any table or script exists inside it — changing it is a migration exercise, not a rename.

What happens if two administrators create tables in the wrong scope by mistake?

ServiceNow allows moving some artifacts between applications after the fact via Studio's application picker, but not all artifact types support this cleanly, and data already inserted against a wrongly scoped table does not move automatically. It is far cheaper to get the scope indicator right before creating anything than to fix it afterward.

Do we need a brand-new scope for every new HR or IT self-service idea?

Not necessarily. Where a new idea is a natural extension of an existing employee-facing application (as Employee Raise Issue is to Employee Center), building inside that existing scope is often the right call, provided the existing application's purpose is not stretched beyond recognition. Where an idea is unrelated in audience or purpose, a new scope is usually cleaner.

Who should be the application owner?

The owner field should name a real accountable individual or team — not a generic "IT" placeholder — since this is who future administrators will look to when deciding whether the application is safe to change, extend, or retire.

14. Summary and Handoff to Phase 2

At the end of this phase, a working, correctly scoped application exists — either newly created or an existing application confirmed suitable for this solution — ready to receive the custom table, business logic, Record Producer, portal pages, and widgets documented in the following phases.

Proceed to Phase 2 — Creating a Custom Table and Fields, which builds the Employee Issue table inside the application scope established here.

14.1 Phase 1 Deliverables Checklist

Deliverable	Status
Application (or existing scope) confirmed and named	Complete
Application description documented	Complete
Naming conventions agreed (section 9)	Complete
Owner assigned	Complete
Ready to receive Phase 2 table	Ready